Hostel Hive

OWASP Top 10 Compliance Evidence

By

Hasinthaka Piyumal

SE/2021/036

Navon Sanjuni

SE/2021/061

Thagya Kavindi

SE/2021/062

Tasheen Darshika

SE/2021/030

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

SENG 34213: System Development Project

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Version No	Date	Author(s)	Summary of Changes
1.0	2026/10/25	Hasinthaka, Sanjuni, Thagya, Tasheen	First version of the OWASP compliance evidence.
1.1	2026/10/25	Hasinthaka, Sanjuni, Thagya, Tasheen	Backend moved to Go 1.27.3 with updated packages. govulncheck is now clean and runs in backend CI. A06 marked as done.
1.2	2026/10/25	Hasinthaka, Sanjuni, Thagya, Tasheen	Added HTTPS evidence for the API on Render and the web app on Vercel. A02 marked as done.
1.3	2026/10/25	Hasinthaka, Sanjuni, Thagya, Tasheen	Backend now logs every refused request. A09 marked as done.

Table 1: Version History

Table of Contents

1 Introduction 1

1.1 Purpose 1

1.2 What We Checked 1

1.3 Link to the SRS 1

1.4 How Our System Handles Security 2

2 Summary 3

3 Evidence for Each Risk 6

3.1 A01: Broken Access Control 6

3.2 A02: Cryptographic Failures 7

3.3 A03: Injection 8

3.4 A04: Insecure Design 9

3.5 A05: Security Misconfiguration 10

3.6 A06: Vulnerable and Outdated Components 11

3.7 A07: Identification and Authentication Failures 13

3.8 A08: Software and Data Integrity Failures 13

3.9 A09: Security Logging and Monitoring Failures 14

3.10 A10: Server-Side Request Forgery 15

4 Things Still To Do 16

5 Threat Model 17

6 References 18

List of Tables

Table 1: Version History II

Table 2: Repositories Checked 1
Table 3: SRS Security Requirements 2

Table 4: OWASP Top 10 Status 3

Table 5: Remaining Work 16

Table 6: STRIDE Threat Model 17

List of Figures

Figure 1: Backend CI run. Firebase rules tests, Go tests and database checks all passed 4
Figure 2: Frontend CI run. Web checks and the dependency security scan passed 4
Figure 3: Mobile app CI run 5
Figure 4: ESP32 firmware CI run 5

Figure 5: Access control and login tests passing on our machine 6
Figure 6: HTTPS checks on the live API on Render 8
Figure 7: HTTPS checks on the web dashboard on Vercel 8
Figure 8: Input checking tests for account management and scanner endpoints 9
Figure 9: Real responses from the backend. Only a short error code is returned 10
Figure 10: Go module check and npm audit 11
Figure 11: govulncheck on the backend before the fix 12
Figure 12: govulncheck on the backend after the fix 12
Figure 13: Log lines written by the backend for refused, allowed and failed requests 15

CHAPTER 1

INTRODUCTION

1.1 Purpose

Section 8.1 of the SENG 34213 project guideline says every team must show how they handled the OWASP Top 10 security risks. This document is our answer to that. It is Deliverable 8 in Section 10.1 of the guideline.

For each risk we write down what the guideline asks for, what we built, and where the proof is. The proof is a screenshot, a test, or a file in one of our GitHub repositories. If something is not finished yet, we say so and list it in Chapter 4.

1.2 What We Checked

We checked these repositories in the `Hostel-Hive` GitHub organisation:

Repository	**What it contains**
hostelhive-backend	Go REST API, PostgreSQL migrations, Firebase rules
hostelhive-frontend	React web dashboard for staff and students
hostelhive-mobileapp	Flutter mobile app for students
hostelhive-hardware	ESP32 QR scanner firmware
hostelhive-infra	Docker Compose and Nginx setup

Table 2: Repositories Checked

Firebase and Cloudflare R2 are outside our scope. Google and Cloudflare are responsible for the security of those services.

1.3 Link to the SRS

The SRS already has three security requirements. This table shows which OWASP risk covers each one.

SRS ID	Requirement	OWASP risk
NFR007	Data in transit is encrypted with TLS 1.2 or HTTPS.	A02
NFR008	Unauthorised access to admin endpoints is always blocked.	A01, A07
NFR009	All inputs are checked to stop SQL injection and XSS.	A03

Table 3: SRS Security Requirements

1.4 How Our System Handles Security

Users log in with Firebase Authentication. The web dashboard and the mobile app send the Firebase login token with every request. The backend checks that token, then loads the user's role from our own database.

There are five roles: admin, warden, sub warden, security staff and student. Each API route lists the roles that can use it. Anyone else gets an error.

The ESP32 QR scanners are not users. Each scanner has its own secret key, and that key only works on the scan endpoints.

The backend API runs on Render at `https://hostelhive-backend-dev.onrender.com`. The web dashboard runs on Vercel at `https://web-kohl-delta-90.vercel.app`. Both are only served over HTTPS.

CHAPTER 2

SUMMARY

The table below shows where we stand on each risk. Every risk is now covered. A few small follow ups are listed in Chapter 4.

ID	Risk	What the guideline asks	Status
A01	Broken Access Control	Role check on every endpoint. Tests show a wrong role gets 403.	**Done**
A02	Cryptographic Failures	Hashed passwords, hashed tokens, TLS.	**Done**
A03	Injection	Parameterised queries and input checks.	**Done**
A04	Insecure Design	Threat model written and used.	**Done**
A05	Security Misconfiguration	No default passwords. No stack traces in errors.	**Done**
A06	Vulnerable Components	Dependency scan in CI. No high or critical issues.	**Done**
A07	Authentication Failures	Lockout after failed logins. Safe sessions.	**Done**
A08	Integrity Failures	Lock files committed. CI checks them.	**Done**
A09	Logging Failures	Log logins, errors and data access. No personal data in logs.	**Done**
A10	SSRF	Check any URL the server fetches.	**Done**

Table 4: OWASP Top 10 Status

Our CI pipelines were passing when we wrote this. The screenshots below show the latest runs.

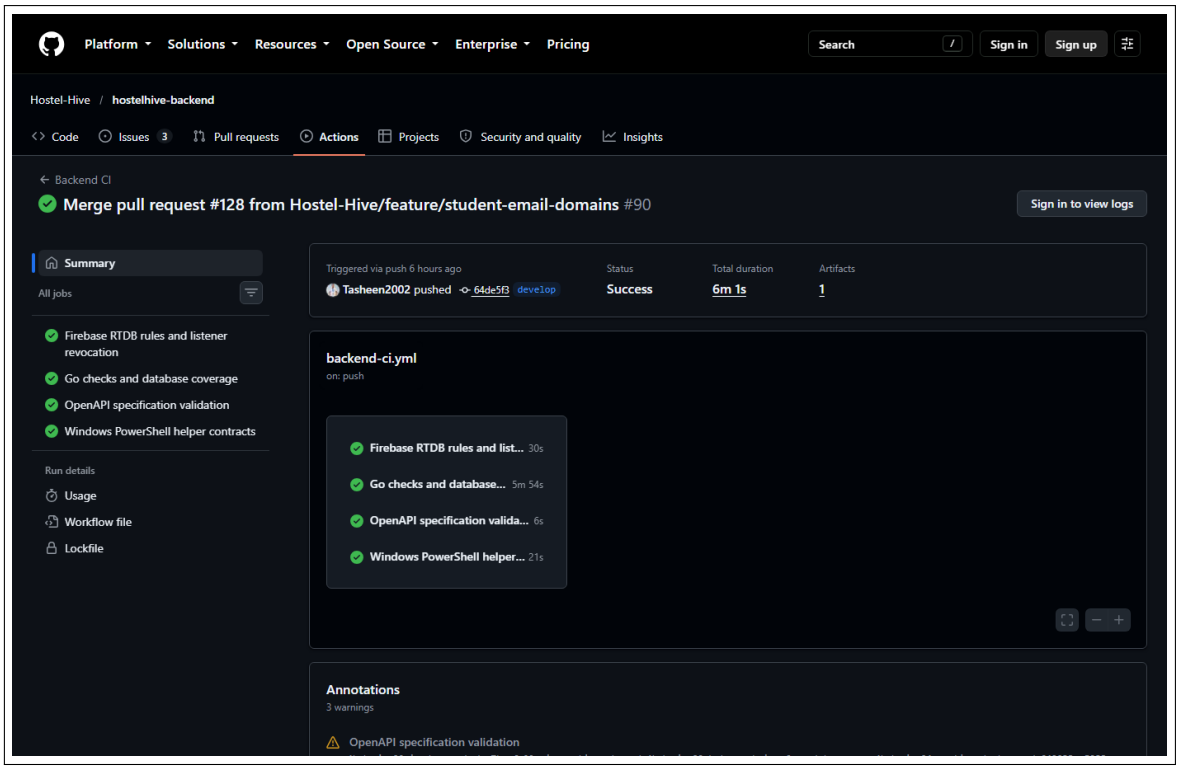


Figure 1: Backend CI run. Firebase rules tests, Go tests and database checks all passed

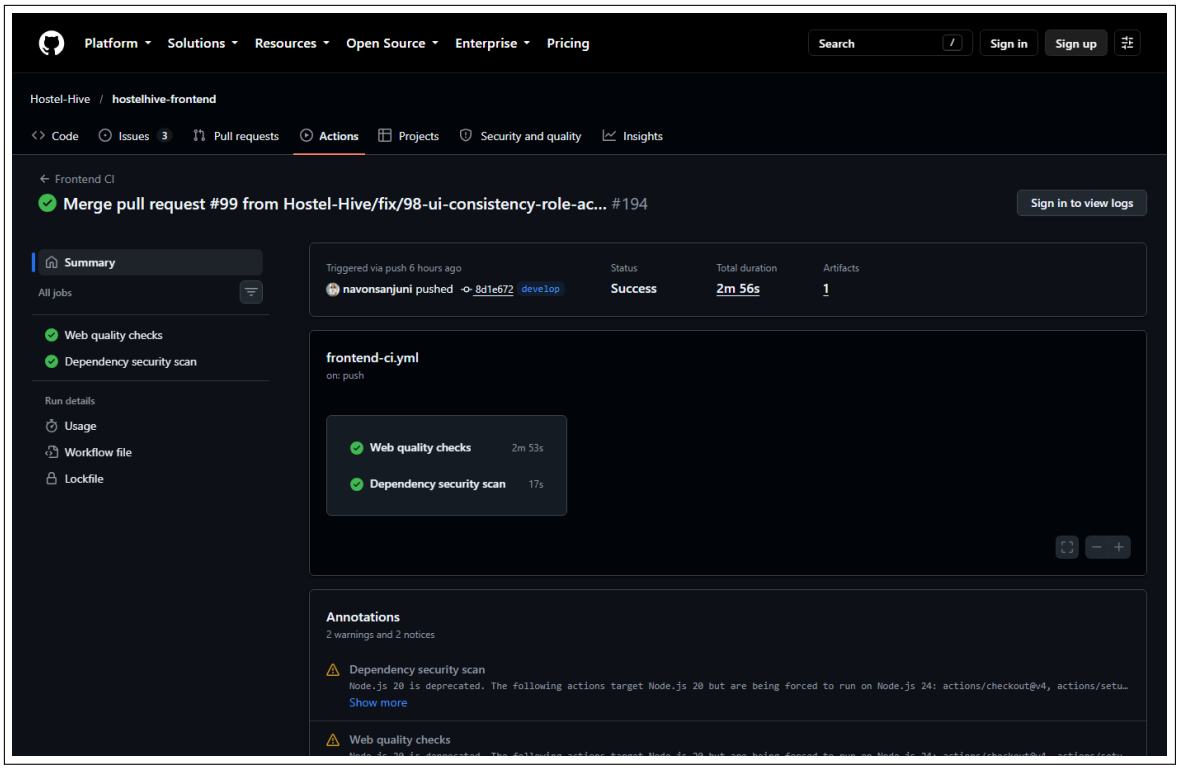


Figure 2: Frontend CI run. Web checks and the dependency security scan passed

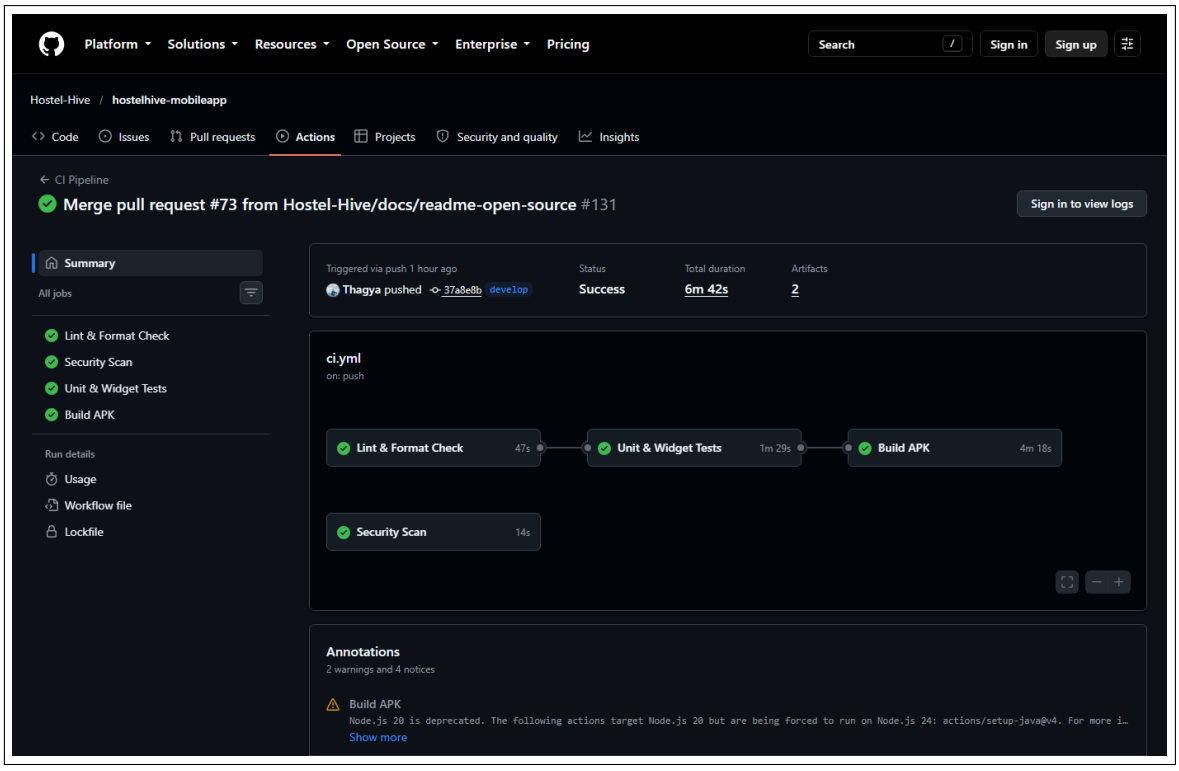


Figure 3: Mobile app CI run

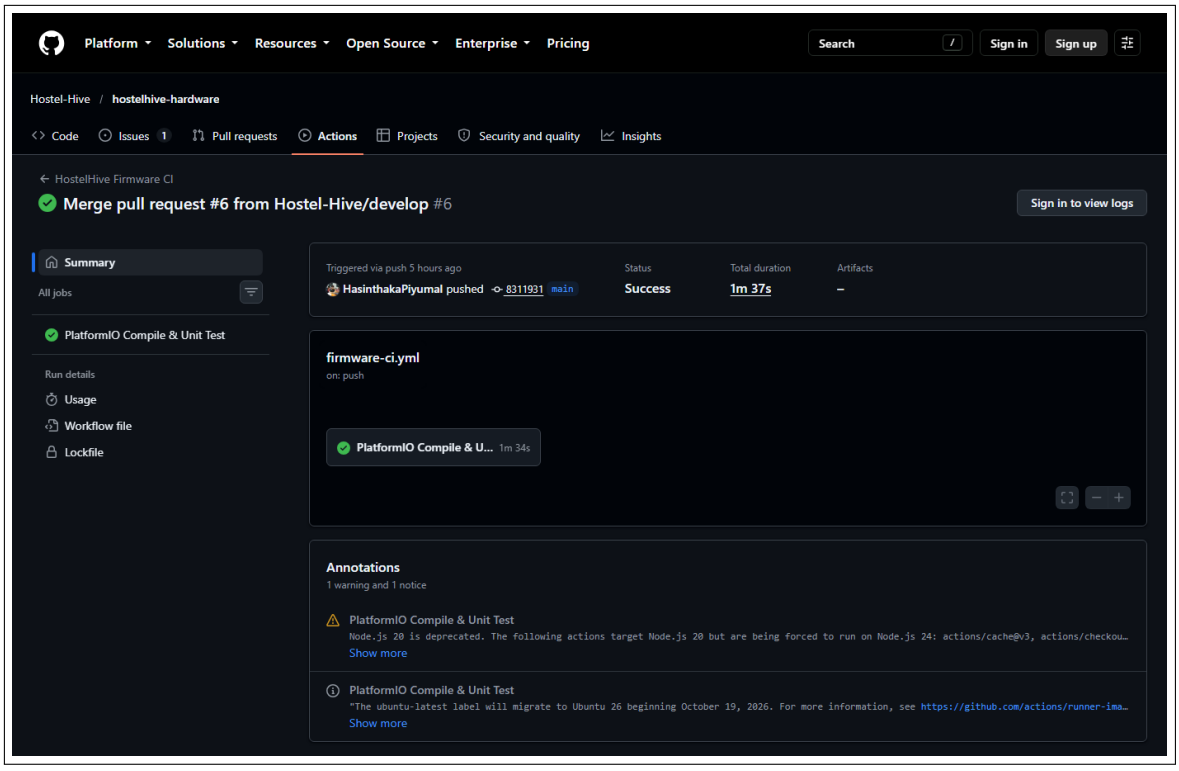


Figure 4: ESP32 firmware CI run

CHAPTER 3

EVIDENCE FOR EACH RISK

3.1 A01: Broken Access Control

What the guideline asks: a role check on every endpoint, and tests that show unauthorised users get a 403 error.

What we did:

- Every protected route goes through two checks. The first checks the login token. The second, `RequireRoles`, checks the user's role against the roles allowed on that route.
- If anything is wrong (no role, unknown role, disabled account), the request is refused. It never lets a request through by mistake.
- We read the role from our database on every request. If an admin changes someone's role or disables them, it works straight away.
- Student routes like `/api/v1/me` only return the logged in student's own data.
- The Firebase Realtime Database blocks all reads by default. Staff get short time access from the server only.

Where to look: `internal/shared/middleware/authorization.go` and `firebase/database.rules.json` in hostelhive-backend.

Proof: the tests in Figure 5 try every role against every type of route. Figure 9 shows a student calling an admin route and getting 403 Forbidden.



Figure 5: Access control and login tests passing on our machine

Status: Done.

3.2 A02: Cryptographic Failures

What the guideline asks: passwords hashed with bcrypt (cost 12), tokens stored as hashes, and TLS turned on.

What we did:

- Our backend never sees passwords. Firebase Authentication handles login and stores passwords with scrypt. OWASP rates scrypt as strong as bcrypt, so we did not add bcrypt ourselves.
- Scanner keys and student invitation tokens are saved as SHA-256 hashes, not as plain text. Scanner keys are compared in constant time.
- Student photos and complaint images sit in a private storage bucket. Only logged in users with the right role can open them.
- The mobile app keeps login data in secure storage (Android Keystore and iOS Keychain).
- Firebase and Cloudflare R2 are only reached over HTTPS.
- Our API is hosted on Render and only works over HTTPS. Render puts a valid certificate on it (issued by Google Trust Services). If someone uses plain `http://`, they get a 301 redirect to `https://`.
- The API accepts only TLS 1.2 and TLS 1.3. We tried TLS 1.0 and 1.1 and the server refused both. This meets SRS requirement NFR007.
- The web dashboard on Vercel also redirects HTTP to HTTPS and sends an HSTS header, so browsers will only use HTTPS for it.

Where to look: `internal/modules/attendance/service/scanner_device_service.go` and `migrations/000025_mobile_push_notifications.up.sql`.

Proof: Figure 6 shows the checks we ran against the live API. Figure 7 shows the same for the web dashboard.

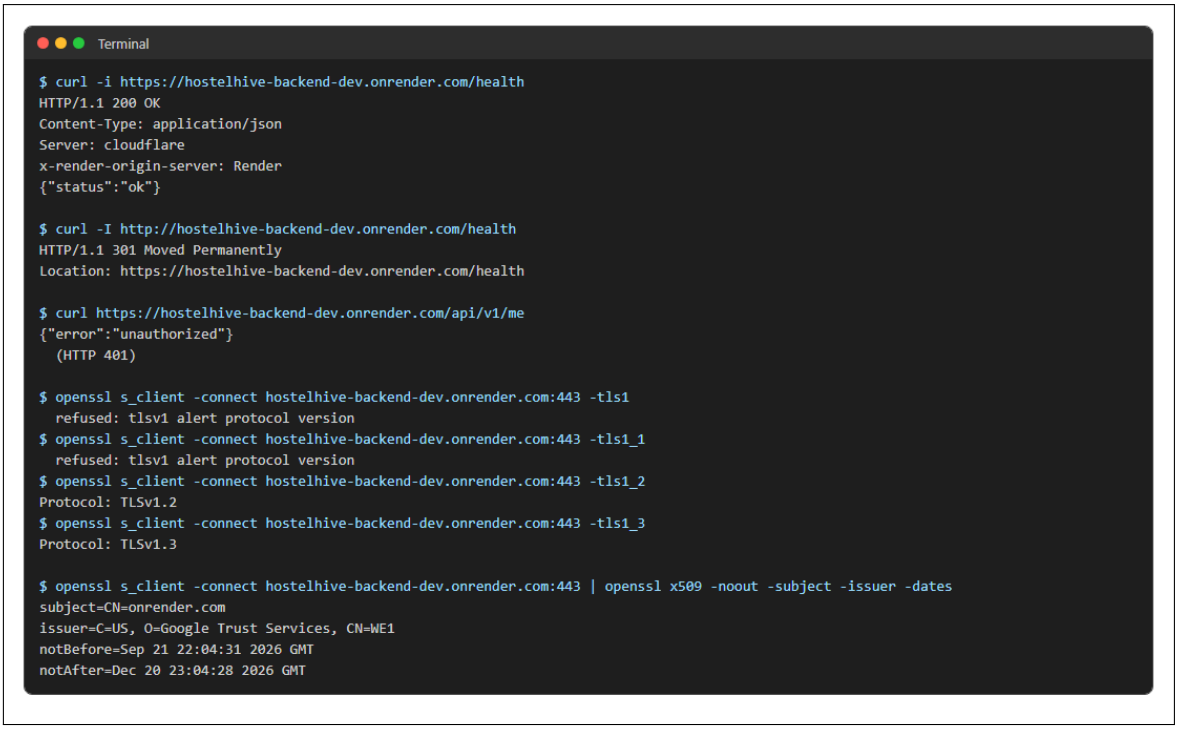


Figure 6: HTTPS checks on the live API on Render

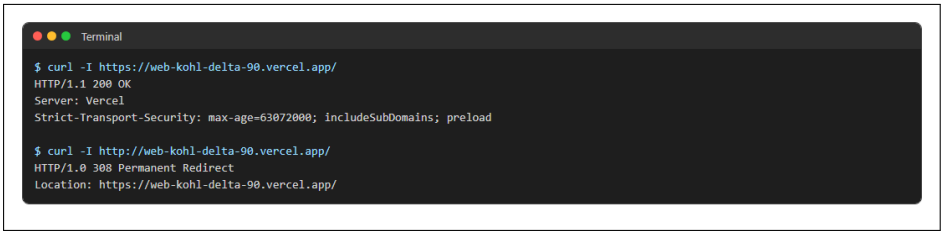


Figure 7: HTTPS checks on the web dashboard on Vercel

Small things left: the API does not send an HSTS header yet, and the ESP32 change that checks the server's root certificate is written but not merged to GitHub yet. Both are listed in Chapter 4.

Status: **Done**.

3.3 A03: Injection

What the guideline asks: parameterised queries or an ORM, and checked and cleaned input.

What we did:

- Every SQL query uses placeholders (`$1`, `$2`) with the pgx driver. We never add user text into a SQL string.

- Search text is escaped, so `%` or `_` typed by a user is treated as a normal character.
- A shared validation package rejects empty text, text that is too long, control characters and badly formed IDs.
- Each endpoint has a size limit on the request body. For example a QR scan can be at most 2 KB.
- Uploaded images are checked before we save them.
- React escapes everything it shows on screen, which stops XSS. We do not use `dangerouslySetInnerHTML` anywhere.

Where to look: `internal/shared/validation/validator.go` and the `repository` folders in each backend module.

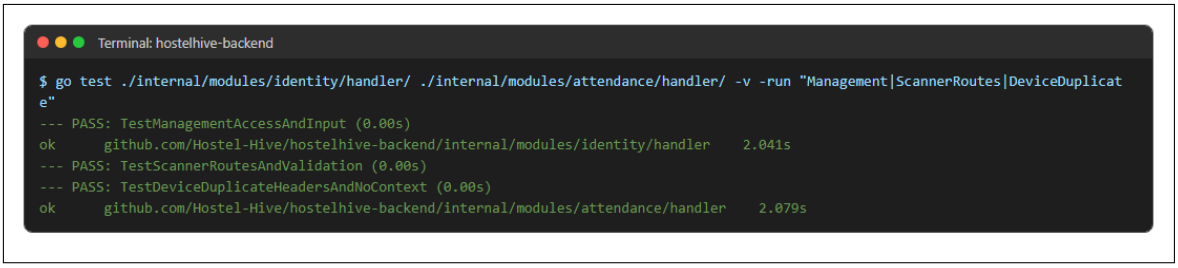


Figure 8: Input checking tests for account management and scanner endpoints

Status: **Done**.

3.4 A04: Insecure Design

What the guideline asks: a threat model in the SDS, and a design that was checked against it.

What we did:

- We wrote a threat model using STRIDE. It is in Chapter 5 of this document, and the SDS security section points to it.
- We blocked some ways people could misuse the system. A QR code scanned twice within a short time only counts once. Late access requests are checked against hostel hours on the server.
- Every pull request needs one review from another team member before it is merged.

Status: **Done**.

3.5 A05: Security Misconfiguration

What the guideline asks: no default passwords, and no stack traces in production error messages.

What we did:

- All error responses come from one function. It only sends a short code like `"forbidden"`. No stack trace or database message reaches the user.
- Error responses have `Cache-Control: no-store`.
- There are no default passwords. `.env.example` only has placeholders like `REPLACE_ME`. Real `.env` files and the firmware `secrets.h` are in `.gitignore`.
- The server will not start if `APP_ENV` has an unknown value.
- The HTTP server has read and write timeouts, so slow clients cannot hold connections open forever.

Proof: Figure 9 shows what the server sends back. In the last case the database error has a password message inside it, but the user only sees `service_unavailable`.

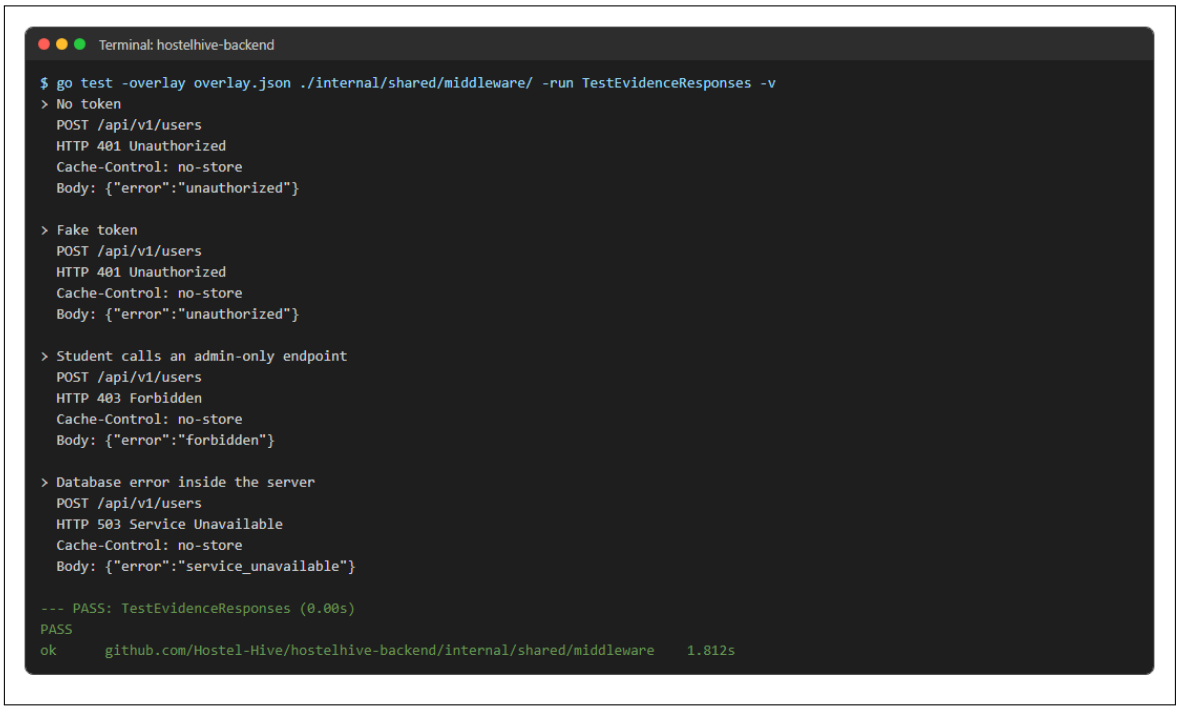


Figure 9: Real responses from the backend. Only a short error code is returned

Status: **Done**.

3.6 A06: Vulnerable and Outdated Components

What the guideline asks: a dependency scan in CI, and no high or critical problems.

What we did:

- The frontend CI runs `npm audit --audit-level=high`. It found no problems (Figure 10).
- The mobile app uses Dependabot, which checks packages every week.
- The backend CI now runs `govulncheck`. It fails the build if our code calls anything with a known vulnerability.

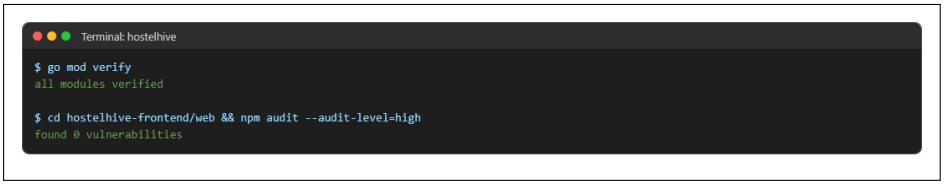


Figure 10: Go module check and npm audit

What we found and fixed: the first time we ran govulncheck on the backend, it reported a long list of known problems in code we actually call (Figure 11). Most were in the Go 1.27.0 standard library. The rest were in `golang.org/x/net`, `golang.org/x/text`, `golang.org/x/image` and `excelize`.

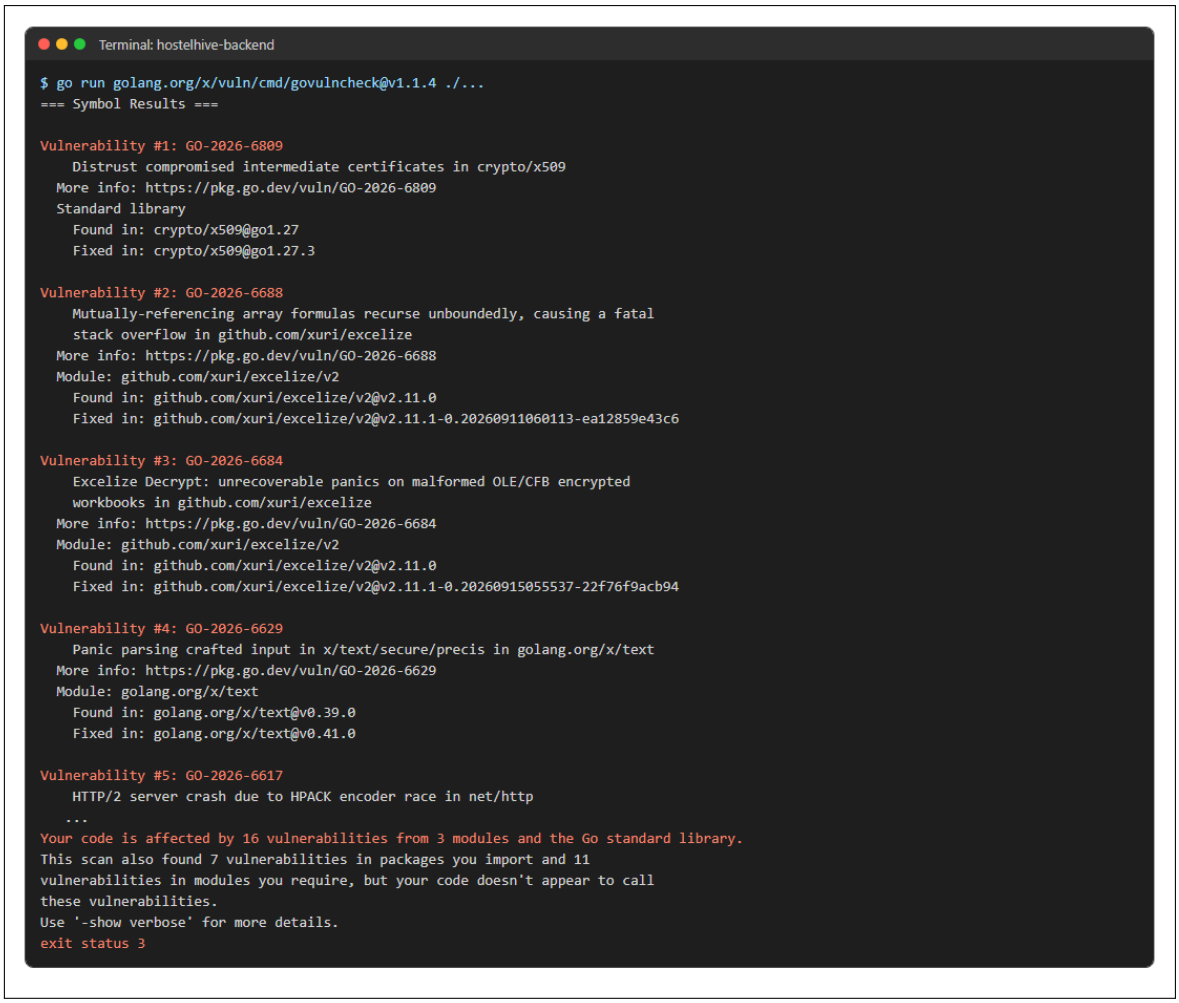


Figure 11: govulncheck on the backend before the fix

We moved the backend to Go 1.27.3 and updated those packages (excelize to the commit that fixes its issues, since there is no new release yet). After that, govulncheck found nothing our code calls (Figure 12). It still lists a few problems in packages we import, but in functions we never call, so they do not affect us.



Figure 12: govulncheck on the backend after the fix

We ran the full backend checks again on the latest `develop` code with the new versions:

migrations up, down and up again, gofmt, `go mod verify`, `go vet`, the build, the OpenAPI check, and the whole test suite with the race detector against PostgreSQL 16. Everything passed. The change is on the branch `fix/security-vulnerabilities`.

Status: **Done**.

3.7 A07: Identification and Authentication Failures

What the guideline asks: lock accounts after a number of failed logins, manage sessions safely, MFA is optional.

What we did:

- Firebase blocks an account for a while after too many wrong passwords. The web app then shows "Too many sign-in attempts. Please try again later."
- Login errors never say whether the email exists.
- The backend only accepts one header in the form `Bearer <token>`. The token is checked by Firebase for signature and expiry.
- When an admin disables an account, we cancel that user's Firebase sessions, so they are logged out.
- New students must finish account setup before they can use any other page.
- We did not add MFA. The guideline says it is optional.

Proof: the login tests in Figure 5 cover missing, fake, expired and revoked tokens. Figure 9 shows a missing token and a fake token both getting 401.

Status: **Done**.

3.8 A08: Software and Data Integrity Failures

What the guideline asks: lock files in the repository, and CI that checks them.

What we did:

- Lock files are committed: `go.sum`, `package-lock.json` and `pubspec.lock`.
- Backend CI runs `go mod verify`. It checks every package against `go.sum` (Figure 10).
- CI installs Node packages with `npm ci`, which fails if the lock file does not match.
- The backend CI pins GitHub Actions to exact commit IDs, not tags that can change.

- CI runs every database migration up, all the way down, and up again on a fresh database.

Status: **Done**.

3.9 A09: Security Logging and Monitoring Failures

What the guideline asks: log login events, errors and data access, without personal data or secrets.

What we did:

- The server writes JSON logs for start, stop, config errors and background job failures.
- Important actions are saved in the database with who did it and when. This covers scanner changes, bed allocations, account creation, account disabling, scans, leave decisions and late access decisions.
- Firebase shows the last sign-in time of every user in its console.
- Every API request now writes one log line when it finishes. The line has the method, the route, the status code, the time it took, and the user or scanner ID if we know who it was.
- Refused requests (401 and 403) are logged as warnings. Server errors (500 and up) are logged as errors. So a wrong role, a fake token or a disabled account is easy to find in the logs.
- We log the route pattern, like `/api/v1/users/{userID}`, not the real path. The query string, headers and body are never logged. This keeps tokens, emails, QR codes and other personal data out of the logs. We only log our internal user ID, not the email or name.
- Successful health checks from Render are skipped so they do not fill the logs. Failed ones are still logged.

Where to look: `internal/shared/middleware/request_log.go` and `request_log_test.go` in hostelhive-backend.

Proof: Figure 13 shows the real log lines for a missing token, a fake token, a student calling an admin route, an admin calling it, and a database failure. The request had an email in the query string, and the database error had a password message in it. Neither shows up in the logs.



Figure 13: Log lines written by the backend for refused, allowed and failed requests

The tests in `request_log_test.go` check the level, route, status and user ID of each line. They also fail if a token, email, query string or database message ever shows up in a log line.

Status: Done.

3.10 A10: Server-Side Request Forgery

What the guideline asks: check any URL from the user and only allow known domains.

What we did:

- No endpoint takes a URL from the user and fetches it. Images are uploaded as files, not as links.
- The server only talks to Firebase, FCM and Cloudflare R2. Their addresses come from our server settings, never from a request.
- Links we send back, like a student's photo link, are built by the server from a checked ID.

So the only places the server can reach are the ones in our settings. If we ever need to fetch a user's URL, we will add a domain allow list first.

Status: Done.

CHAPTER 4

THINGS STILL TO DO

These items came up while we wrote this document. We plan to finish the high priority ones before the final demo.

No	Risk	Task	Repository	Priority
1	A06, A09	Merge the backend branch with the Go 1.27.3 upgrade, the govulncheck CI step and the request log (ready, not merged yet).	backend	High
2	A06	Move excelize to a tagged release once one includes the fix.	backend	Low
3	A02	Merge the ESP32 HTTPS and root certificate change.	hardware	High
4	A02	Send an HSTS header from the API.	backend	Low
5	A05	Add security headers (CSP, X-Frame-Options) for the web dashboard in the Vercel settings.	frontend	Medium
6	A06	Turn on Dependabot for backend, frontend and firmware.	all	Medium
7	A07	Optional: MFA for admin accounts.	frontend	Low

Table 5: Remaining Work

CHAPTER 5

THREAT MODEL

We used STRIDE to list what could go wrong and how we stop it.

Type	**Threat**	**How we stop it**
Spoofing	Someone uses a fake or stolen login token.	Firebase checks every token. Disabled users are logged out.
Spoofing	A fake scanner sends attendance scans.	Each scanner has its own secret key, stored as a hash.
Tampering	SQL injection through a search box or form.	Placeholders in every query, input checks, size limits.
Tampering	A student scans twice or shares a QR code.	Repeat scans in a short time are ignored. Every scan saves the scanner and time.
Repudiation	A staff member says they did not approve a leave.	Decisions are saved with the user and time.
Information disclosure	A student opens another student's data.	Role checks on the server. `/me` routes only return your own data.
Information disclosure	Error messages show internal details.	Errors only return a short code.
Information disclosure	Secrets get pushed to GitHub.	`.env` and `secrets.h` are ignored. Examples only have placeholders.
Denial of service	Huge or very slow requests.	Body size limits and server timeouts.
Denial of service	Password guessing on staff accounts.	Firebase blocks the account after too many tries.
Elevation of privilege	A student calls an admin endpoint.	`RequireRoles` returns 403. Covered by tests.
Elevation of privilege	A bad package or CI action.	Lock files, `go mod verify`, `npm ci`, pinned actions.

Table 6: STRIDE Threat Model

CHAPTER 6

REFERENCES

[1] OWASP Foundation, *OWASP Top 10:2021*, https://owasp.org/Top10/

[2] OWASP Foundation, *Password Storage Cheat Sheet*, https://cheatsheetseries.owasp.org/

[3] Google, *Firebase Authentication*, https://firebase.google.com/docs/auth

[4] Go Team, *govulncheck*, https://go.dev/doc/security/vuln/

[5] Software Engineering Teaching Unit, University of Kelaniya, *SENG 34213 System Development Project Guideline*, Chapter 8.

[6] HostelHive Team, *Hostel Hive Software Requirement Specification*, Version 1.0, 2026.